P3103R2
More bitset operations

Published Proposal, 2024-03-07

This version:
https://eisenwave.github.io/cpp-proposals/more-bitset-operations.html

Author:
Jan Schultke

Audience:
SG18, LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Source:
eisenwave/cpp-proposals

Abstract

This paper proposes adding a counterpart `std::bitset` member function for the utility functions in `<bit>`.

Table of Contents

§ 1. Revision history

§ 1.1. Changes since R1

- Mentioned [P3104R2], now that it has been published.
- Greatly expanded § 4 Design considerations based on feedback from LEWGI.

§ 1.2. Changes since R0

- Simplified proposed wording by defining `rotl` as an equivalence in terms of `rotr`.
- Fixed minor editorial mistakes.

§ 2. Introduction

[P0553R4] added the bit manipulation library to C++20, which introduced many useful utility functions.

Some of these already have a counterpart in `std::bitset` (such as `popcount` as `bitset::count`), but not nearly all of them. This leaves `bitset` overall lacking in functionality, which is unfortunate because `std::bitset` is an undeniably useful container. At the time of writing, it is used in 73K files on GitHub; see [GitHub1].

`std::bitset` does not (and should not) expose the underlying integer sequence of the implementation. Therefore, it is not possible for the user to implement these operations efficiently themselves.

§ 3. Proposed changes

For each of the functions from the bit manipulation library that are not yet available in `std::bitset`, add a member function. Add a small amount of further useful member functions.

<bit> function	**Proposed `bitset` member**
`std::has_single_bit(T)`	`one()`
`std::countl_zero(T)`	`countl_zero()`
	`countl_zero(size_t)`
`std::countl_one(T)`	`countl_one()`
	`countl_one(size_t)`
`std::countr_zero(T)`	`countr_zero()`
	`countr_zero(size_t)`
`std::countr_one(T)`	`countr_one()`
	`countr_one(size_t)`
`std::rotl(T, int)`	`rotl(size_t)`
`std::rotr(T, int)`	`rotr(size_t)`
	`reverse()`

The additional overloads for the counting functions allow counting from a starting position. This can be useful for iterating over all set bits:

```
bitset<128> bits;
for (size_t i = 0; i != 128; ++i) {
    i += bits.countr_zero(i);
```

```
    if (i == 128) break;
    // ...
}
```

> NOTE: `byteswap` and `bit_cast` counterparts are not proposed, only functions solely dedicated to the manipulation of bit sequences.

§ 4. Design considerations

The names and signatures of the proposed member functions is a mixture of `<bit>` function templates and the existing conventions of `bitset`. Sadly, it is impossible to be consistent with both parts of the standard library. For example, `<bit>` functions typically prefer `int` parameters for offsets, whereas `bitset` uses `size_t` virtually everywhere. `bitset` also has a very brief convention of `none()` or `all()`, where `<bit>` has a `has_single_bit` function.

This proposal generally prefers local consistency, i.e. it is more important to be consistent with `bitset` than functions from a different part of the standard library, even if those distant functions are similar in purpose.

§ 4.1. `reverse`

`reverse` is a novel invention, not taken from `<bit>`. However, a `bit_reverse` counterpart is proposed in [P3104R2] "Bit permutations".

This function is added because the method for reversing integers may be tremendously faster than doing so bit by bit. ARM has a dedicated `RBIT` instruction for reversing bits, which could be leveraged.

§ 4.1.1. In-place reversal vs. reverse-copy

I propose `reverse` to modify a bitset in-place rather than creating a reversed copy. The interface of `bitset<N>` can already accommodate large `N`, and use cases with large `N` exist.

Compilers also should not be expected to find the optimization from a `std::ranges::reverse_copy` with later assignment to an in-place `std::reverse`.

EXAMPLE 1 ¶

In [CompilerExplorer1], Clang does not find the optimization from:

```
void reverse_in_place(bitset& x) {
    x = x.reverse_copy();
}
```

... to the in-place counterpart for a 4096-bit `bitset`. Instead, it allocates 512 bytes of stack space, performs a reverse-copy, and `memcpy`s the result back into `x`.

I believe that all operations of a `bitset<N>` with large `N` should remain usable, and copying the whole set for an operation may be unacceptable due to danger of overflowing the stack and/or performance considerations.

A perfect optimization would be very difficult due to substantial differences between the `reverse` and `reverse_copy` algorithms.

§ 4.1.2. `reverse(bitset)`

Instead of a `reverse` member function, it would also be possible to add an overload for the `reverse` algorithm which accepts a `bitset`. There exists precedent (e.g. `std::move`), although this is not the first design which comes to mind.

I do not oppose this idea; LEWG needs to vote on this.

§ 4.2. Common interface with `<bit>`

Another considered alternative is the creation of a common interface with the function templates of `<bit>`. This would mean adding overloads for say, `countl_zero` and other functions which would accept `bitset` in addition to unsigned integers.

This proposal does not pursue this design for multiple reasons:

1. The use case of this is unclear. I have personally never needed to write generic code which does bit manipulation on either integers or bitsets, and don't know of any code base where this is done.
2. There is too much friction in the design of `bitset` and `<bit>`, such as the use of `size_t` vs. `int`, the use of exceptions vs. preconditions, different naming schemes, etc.

3. The functions in `<bit>` are all pure, but for `bitset<N>`, copying it for the purpose of function calls and returning modified copies becomes impractical for large `N`. See also § 4.1.1 In-place reversal vs. reverse-copy.
4. A common interface does not eliminate the need for new member functions. It would be very surprising to users if some functionality was only available through `<bit>` (e.g. `countr_zero`) whereas other functionality was available both as a member function and as a `<bit>` function (e.g. `popcount`/`.count()`). Such a design would be inconsistent, and nonsensical when taken out of its historical context.
5. A future proposal could still create a common interface for `<bit>` and `bitset`. The addition of member functions adds no obstacles in this regard.

Also note that most of these issues equally apply to sharing a common interface with `std::simd`.

§ 4.3. Counting overloads

`countl_zero`, `countr_zero`, `countl_one`, and `countr_one` are overloaded member functions which take a `size_t` argument or nothing.

This is preferable to a single member function with a defaulted argument because the overloads have different `noexcept` specifications. The overloads which take no arguments have a wide contract and can be marked `noexcept`, whereas the overloads taking `size_t` may throw `out_of_range`.

§ 4.3.1. Alternative designs facilitating iteration

Besides the proposed design for `countl_zero`, there are at least two alternatives, mainly motivated by simplifying iteration. As mentioned above, we can iterate over all one-bits of a `bitset` as possible.

```
bitset<128> bits;
for (size_t i = 0; i != 128; ++i) {
    i += bits.countr_zero(i);
    if (i == 128) break;
    // ...
}
```

LEWG should vote on whether to go ahead with the current design or prefer one of the alternatives below.

4.3.1.1. Infallible `countr_zero`

If `countr_zero` could not throw but returned zero on an index out of bounds, this could be shortened to:

```
bitset<128> bits;
for (size_t i = 0; i += bits.countr_zero(i) != 128; ++i) {
    // ...
}
```

An infallible `countr_zero` would also cater to users who do not want to use exceptions, whereas the current proposal perpetuates the existing exception-based design.

4.3.1.2. `find_next_one`

It would be possible to add a member function which increments and advances to the next bit in a one operation.

```
bitset<128> bits;
for (size_t i = 0; (i = bits.find_next_one(i)) != bitset<N>::npos; ) {
    // ...
}
```

A possible issue is that this operation is extremely specialized towards iteration.

As with `countr_zero`, it also needs be considered whether `find_next_one` should handle a bad index by throwing an exception, if LEWG prefers this kind of function.

4.3.1.3. No novel overloads

It is also possible to drop the overloads taking `size_t` from this proposal. Perhaps a future proposal focused on iteration for `bitset` could approach the issue of facilitating iteration. This was most recently attempted in [P0161R0].

However, such functions are well within the scope of this proposal, especially if they share a name with other proposed members, and they do not pose a substantial design/wording hurdle. In short, we may as well do it.

§ 4.3.2. New overloads for `<bit>`?

The overloads taking `size_t` don't need a counterpart in `<bit>` because they are very easy to emulate:

```
int countl_zero(std::unsigned_integral auto x, int off) {
    return std::countl_zero(x >> off);
}
```

I consider these too trivial to justify adding such convenience functions to `<bit>`, even if it would create more symmetry between `bitset` and `<bit>`.

§ 5. Impact on existing code

The semantics of existing `bitset` member functions remain unchanged, and no existing valid code is broken.

This proposal is purely an extension of the functionality of `bitset`.

§ 6. Implementation experience

[Bontus1] provides a `std::bitset` implementation which supports most proposed features. There are no obvious obstacles to implementing the new features in common standard library implementations.

§ 7. Proposed wording

The proposed changes are relative to the working draft of the standard as of [N4917].

Modify the header synopsis in subclause 17.3.2 [version.syn] as follows:

```
#define __cpp_lib_bitset                              ~~202306L~~20XXXXL // als
```

Modify the header synopsis in subclause 22.9.2.1 [template.bitset.general] as follows:

```
namespace std {
  template<size_t N> class bitset {
  public:
    // bit reference
    [...]

    // [bitset.members], bitset operations
    constexpr bitset& operator&=(const bitset& rhs) noexcept;
    constexpr bitset& operator|=(const bitset& rhs) noexcept;
    constexpr bitset& operator^=(const bitset& rhs) noexcept;
    constexpr bitset& operator<<=(size_t pos) noexcept;
    constexpr bitset& operator>>=(size_t pos) noexcept;
    constexpr bitset  operator<<(size_t pos) const noexcept;
    constexpr bitset  operator>>(size_t pos) const noexcept;
+   constexpr bitset& rotl(size_t pos) noexcept;
+   constexpr bitset& rotr(size_t pos) noexcept;
+   constexpr bitset& reverse() noexcept;
    constexpr bitset& set() noexcept;
    constexpr bitset& set(size_t pos, bool val = true);
    constexpr bitset& reset() noexcept;
    constexpr bitset& reset(size_t pos);
    constexpr bitset  operator~() const noexcept;
    constexpr bitset& flip() noexcept;
    constexpr bitset& flip(size_t pos);

    // element access
    [...]

    // observers
    constexpr size_t count() const noexcept;
+   constexpr size_t countl_zero() const noexcept;
+   constexpr size_t countl_zero(size_t pos) const;
+   constexpr size_t countl_one() const noexcept;
+   constexpr size_t countl_one(size_t pos) const;
+   constexpr size_t countr_zero() const noexcept;
+   constexpr size_t countr_zero(size_t pos) const;
+   constexpr size_t countr_one() const noexcept;
+   constexpr size_t countr_one(size_t pos) const;
    constexpr size_t size() const noexcept;
    constexpr bool operator==(const bitset& rhs) const noexcept;
    constexpr bool test(size_t pos) const;
```

```
    constexpr bool all() const noexcept;
    constexpr bool any() const noexcept;
    constexpr bool none() const noexcept;
+   constexpr bool one() const noexcept;
  };

  // [bitset.hash], hash support
  template<class T> struct hash;
  template<size_t N> struct hash<bitset<N>>;
}
```

Modify subclause 22.9.2.3 [bitset.members] as follows:

> `constexpr bitset operator>>(size_t pos) const noexcept;`
>
> *Returns*: `bitset(*this) >>= pos`.
>
> `constexpr bitset& rotl(size_t pos) noexcept;`
>
> *Effects*: Equivalent to `rotr(N - pos % N)`.
>
> `constexpr bitset& rotr(size_t pos) noexcept;`
>
> *Effects*: Replaces each bit at position `I` in `*this` with the bit at position `(static_cast<U>(pos) + static_cast<U>(I)) % N`, where `U` is a hypothetical unsigned integer type whose width is greater than the width of `size_t`.
>
> *Returns*: `*this`.
>
> `constexpr bitset& reverse() noexcept;`
>
> *Effects*: Replaces each bit at position `I` in `*this` with the bit at position `N - I - 1`.
>
> *Returns*: `*this`.
>
> [...]
>
> `constexpr size_t count() noexcept;`
>
> *Returns*: A count of the number of bits set in `*this`.
>
> `constexpr size_t countl_zero(size_t pos) const;`
>
> *Returns*: The number of consecutive zero-bits in `*this`, starting at position `pos`, and traversing `*this` in decreasing position direction.
>
> *Throws*: `out_of_range` if `pos` does not correspond to a valid bit position.
>
> `constexpr size_t countl_zero() const noexcept;`
>
> *Returns*: `countl_zero(N - 1)`.

`constexpr size_t countl_one(size_t pos) const;`

Returns: The number of consecutive one-bits in `*this`, starting at position `pos`, and traversing `*this` in decreasing position direction.

Throws: `out_of_range` if `pos` does not correspond to a valid bit position.

`constexpr size_t countl_one() const noexcept;`

Returns: `countl_one(N - 1)`.

`constexpr size_t countr_zero(size_t pos) const;`

Returns: The number of consecutive zero-bits in `*this`, starting at position `pos`, and traversing `*this` in increasing position direction.

Throws: `out_of_range` if `pos` does not correspond to a valid bit position.

`constexpr size_t countr_zero() const noexcept;`

Returns: `countr_zero(0)`.

`constexpr size_t countr_one(size_t pos) const;`

Returns: The number of consecutive one-bits in `*this`, starting at position `pos`, and traversing `*this` in increasing position direction.

Throws: `out_of_range` if `pos` does not correspond to a valid bit position.

`constexpr size_t countr_one() const noexcept;`

Returns: `countr_one(0)`.

[...]

`constexpr bool none() const noexcept;`

Returns: `count() == 0`.

`constexpr bool one() const noexcept;`

Returns: `count() == 1`.

NOTE: The use of a hypothetical integer type in the specification of `rotl` and `rotr` is necessary because `(I + pos) % N` would be incorrect when `I + pos` wraps.

§ References

§ Normative References

[N4917]
Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 5 September 2022. URL: https://wg21.link/n4917

§ Informative References

[Bontus1]
Claas Bontus; d-xo; zencatalyst. *bitset2: bitset improved*. URL: https://github.com/ClaasBontus/bitset2

[CompilerExplorer1]
Jan Schultke. *Example of in-place reversal vs. reverse-copy optimization*. URL: https://godbolt.org/z/KrPsrh9bM

[GitHub1]
GitHub Code Search. *std::bitset language:c++*. URL: https://github.com/search?type=code&auto_enroll=true&q=%22std%3A%3Abitset%22+language%3Ac%2B%2B&p=1

[P0161R0]
Nathan Myers. *Bitset Iterators, Masks, and Container Operations*. 12 February 2016. URL: https://wg21.link/p0161r0

[P0553R4]
Jens Maurer. *Bit operations*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0553r4.html

[P3104R2]
Jan Schultke. *Bit permutations*. 5 April 2024. URL: https://wg21.link/p3104r2